

Talking Script: Software Architecture for Essential & Accidental Uncertainty

15-Minute Presentation Script with Timings (Short Version)

Total Time: 15 minutes (900 seconds)

Slide 1: Title Slide

Time: 0:00 - 0:15 (15 seconds)

Good morning. I'm George Rudolph from Utah Valley University in Orem, Utah, and I'm here today to present work by Neil Harrison, Peter Aldous and myself on the topic of Software Architecture for Essential & Accidental Uncertainty. This work addresses a critical gap in how we think about uncertainty in software systems, and provides a framework for architects to make better design decisions.

Slide 2: The Challenge: From Deterministic to Uncertain

Time: 0:15 - 1:00 (45 seconds)

Let me start with some context. The ENIAC, shown here, was the first general-purpose computer, built in 1945, that solved a large class of numerical problems. The math problems it computed were precisely specified by well-defined inputs, clear algorithms, and deterministic outputs.

But computing has evolved dramatically since then. Over time, capabilities expanded from business applications like spreadsheets to data communications, control systems, games and various waves of AI. As this happened, inputs, outputs, and processes became more varied and less precise. In many ways, challenges are an artifact of modeling non-mathematical problems via mathematical abstractions. We must now deal with uncertainty—and understand how to approach it in architecture and design.

Slide 3: Current State of the Art

Time: 1:00 - 1:25 (25 seconds)

In 2010, David Garlan called for uncertainty to be a first-class concern in software design and development. However, current research and practice primarily focus on fault tolerance and related tactics. While important, this misses a large part of uncertainty and motivates the deeper investigation we outline today.

Slide 4: Essence vs. Accident (Brooks, 1986)

Time: 1:25 - 2:05 (40 seconds)

Our work borrows a fundamental distinction made by Fred Brooks in his famous 1986 paper, “No Silver Bullet.” Brooks distinguished between accidental complexity and essential complexity. Accidental complexity is created by engineering choices and can be mitigated by better tools and abstractions. Essential complexity is inherent in the problem and cannot be removed.

Just as there is essential and accidental complexity, there is essential and accidental uncertainty. We think this distinction is crucial for making appropriate architectural decisions.

Slide 5: Essential and Accidental Uncertainty

Time: 2:05 - 2:40 (35 seconds)

Let me take a moment to define these terms clearly. Accidental uncertainty can be separated from the problem—a transformation exists to remove it from the problem. Essential uncertainty, on the other hand, must be addressed as part of the problem. No transformation can remove it; it’s inherent to what we’re trying to accomplish.

Computing is fundamentally the processing of data. Even programs are data. So we consider uncertainty across three dimensions: input, transformation, and output. Input uncertainty can be essential or accidental. The transformation from input to output can also be essential or accidental. Output uncertainty is often essential, and this is where things get architecturally interesting.

Slide 6: Uncertainty in the Transformation (Algorithms)

Time: 2:40 - 3:35 (55 seconds)

The algorithm or processing logic that transforms input to output can introduce uncertainty. Let’s break this down.

Accidental transformation uncertainty can be fixed. This includes hardware glitches, compiler differences, and implementation bugs. These are engineering problems with engineering solutions.

Essential transformation uncertainty is more fundamental. Sometimes no deterministic algorithm exists to compute the correct output. Image recognition is a classic example—there’s no deterministic formula that says “this is a cat.”

Then there’s essential but intractable uncertainty. The algorithm is computationally infeasible, so approximations introduce uncertainty in accuracy or precision. Examples include the game of Go or many optimization problems.

The critical insight here is that essential transformation uncertainty often reflects uncertain output—and this requires probabilistic or AI-based approaches.

Slide 7: Essential Output Uncertainties

Time: 3:35 - 4:35 (60 seconds)

Let me walk you through the spectrum of essential output uncertainties. At one end, there is a known right answer—mathematical computation. Then, the right answer can be known—like asking how well a product sold on a sales website.

Moving along the spectrum, the right answer exists but is infeasible to compute directly—the game of Go has an astronomical state space. Then we have unknown right answers where we can guess—image recognition falls here.

Further along, the effect is unknown but can be measured—optimization problems. Then the effect is unknown and the system is chaotic—weather forecasting. There’s no specific answer, but we can observe the “right” direction—advertising and many simulation problems.

At the far end, there’s no single right answer because the problem is vague—automated prose writing. And finally, the problem itself is unknown—grounded theory.

This spectrum is crucial because different positions require fundamentally different architectural approaches.

Slide 8: Spectrum of Uncertainty

Time: 4:35 - 4:55 (20 seconds)

This diagram visualizes the spectrum we just discussed. As you can see, uncertainty ranges from deterministic problems with known answers all the way to problems where the problem itself is unknown.

Slide 9: Architectural Approaches

Time: 4:55 - 5:30 (35 seconds)

Different levels and characteristics of uncertainty suggest different architectural styles, patterns, and tactics. When there’s virtually no uncertainty, batch processing works well. For accidental uncertainty, we can use pipes and filters, layers, or broker patterns.

For greater essential uncertainty, we need AI-based approaches: blackboard architectures, neural networks, machine learning, and deep learning. The key is matching the architecture to the nature of the uncertainty.

Slide 10: Spectrum of Architectural Approaches

Time: 5:30 - 5:45 (15 seconds)

This diagram maps architectural approaches to the uncertainty spectrum. As uncertainty increases, we move from traditional deterministic architectures to probabilistic and AI-based systems.

Slide 11: An Architecture Process for Uncertainty

Time: 5:45 - 6:45 (60 seconds)

We propose a five-step architecture process for dealing with uncertainty. First, identify uncertainties associated with the system. Second, for each uncertainty, determine if it is essential or accidental. Third, study the source of uncertainty—external sources are often accidental. Fourth, determine if a transformation can remove the uncertainty. Fifth, if so, apply fault-tolerance and related techniques. If not, you’re dealing with essential uncertainty and need different approaches.

When analyzing uncertainties, ask key questions. For input: Is it due to imprecise or incomplete specification? Is it due to variability in data—transmission issues, user errors, and so on? What can and cannot be known about input at runtime?

For output: What is the objective of the system? How do we know it meets the objective? Is there a known correct output? Is the correct output unknowable but estimable? Can output be deterministically derived from inputs? Are there multiple correct outputs, and if so, is one preferable?

For transformation: Is the relationship between input and output deterministic? If deterministic, is it tractable? These questions help you classify the uncertainty and choose appropriate tactics.

Slide 12: Case Study: Helicopter Rotor Balancing

Time: 6:45 - 7:15 (30 seconds)

Let me illustrate this with a real case study: helicopter rotor balancing. If the main rotor is not perfectly balanced, excess vibration occurs. This vibration causes wear, loss of power, and can compromise safety. The rotor is a complex system with multiple adjustment points, and the relationships between adjustments and outcomes are not straightforward.

For the uncertainty analysis: The input is known and precisely measured—vibration measurements, blade positions, adjustment points. User entry errors are external and accidental;

we apply standard Fault Tolerance techniques to correct this accidental uncertainty. However, each helicopter has unique wear patterns—this is essential uncertainty that must be considered.

For output: The objective is measurable—change in vibration, measured in inches per second, or IPS. The desired output is a range—lower is better. The acceptable standard is 0.5 IPS. But is there a single solution? There may be different sequences of adjustments that achieve the goal, so we have some essential uncertainty here.

For processing: Is the relationship between input and output deterministic? Initially, this was unknown. Combinations of adjustments appeared non-deterministic or even chaotic. The initial approach was to use a neural network to derive suggested solutions, treating this as essential uncertainty requiring AI-based methods.

Slide 13: Case Study: Helicopter Rotor Balancing (continued)

Time: 7:15 - 7:55 (40 seconds)

Further analysis revealed something interesting. Each helicopter has a unique wear profile. The initial neural network was trained on many helicopters, creating a generic approximation. But the adjustment and response history of testing reveals unique profiles. The intermediate result is that there exists a deterministic, repeatable sequence of adjustments with feedback loops. The details are proprietary, but this shows that some of what appeared essential was actually accidental, solvable with the right approach.

The relevant results are impressive. The system reliably balances main rotors to 0.05 IPS—that’s ten times better than the standard of 0.5 IPS. It’s actively marketed to the US Army and other organizations. This demonstrates the importance of understanding essential versus accidental uncertainty—to choose the right architectural approach. Properly classifying uncertainty led to a much better solution.

Slide 14: Case Study: Online Bookstore Evolution

Time: 7:55 - 8:30 (35 seconds)

Let’s look at another case study: an online bookstore. Initially, inputs and outputs were well-specified, with numerous accidental uncertainties like unreliable networks. The architecture was straightforward.

But then they added a “related books” feature. Which books are related? This is an essential uncertainty—the output cannot be known beforehand, and there’s no single right answer. The approaches become probabilistic, requiring recommendation algorithms and similarity metrics.

The evolution continued with using browsing history to promote products. This introduces even more uncertainty; the problem is less well-defined. This requires machine learning approaches and maps to a different position on the uncertainty spectrum. Recommendation engines are a popular, common solution for this type of problem.

Slide 15: References

Time: 8:30 - 8:45 (15 seconds)

Our work is published in HICSS 2026, and we also presented related work at the Intermountain Conference on Engineering, Technology and Computing in 2025. The full paper provides more detail on the framework and additional case studies.

Slide 16: Thank you!

Time: 8:45 - 15:00 (375 seconds, including Q&A)

Thank you for your attention. To summarize: We've presented a framework for understanding essential versus accidental uncertainty in software architecture. This distinction is crucial for selecting appropriate architectural approaches. Essential uncertainty requires probabilistic and AI-based methods, while accidental uncertainty can be addressed with traditional fault-tolerance techniques. The key is proper analysis and classification.

I'm happy to take questions now.

Notes for Practice:

- **Pacing:** Average 60 seconds per slide, but adjust based on complexity
- **Transitions:** Use phrases like “Let me illustrate,” “Moving along,” “The key insight is”
- **Emphasis:** Stress “essential” vs “accidental” throughout
- **Eye contact:** Look at audience, not just slides
- **Case studies:** These are concrete examples—speak clearly and confidently
- **Time buffer:** Built in 6+ minutes at end for questions; adjust if needed
- **Visual slides:** Slides 8 and 10 are primarily visual—speak briefly and let the diagram do the work